

Slide-by-slide voiceover

Title — Obsidian Second Brain

"Quick recap of where we are: this is Obsidian Second Brain — a self-updating knowledge graph over my notes that doesn't just search text, it reasons about what's still true. Phase 1 was a Python CLI pipeline that could read a vault, extract facts with a local LLM, and write them into Neo4j. This is Phase 2: taking that same pipeline and porting it into a native Obsidian plugin, so it runs right where I already write, with zero paid APIs."

01 — Architecture

"Here's the shape of it: one pipeline, three systems. Notes get read straight out of the Obsidian vault, chunked by heading, sent to Ollama running locally for entity and relationship extraction, and merged into a Neo4j graph — all entirely on-device, nothing leaves my machine."

02 — Starting point

"I wasn't starting from scratch. The Python pipeline already worked end to end from the terminal — ingest, extract, store — backed by seven passing tests. So the goal for this phase wasn't to redesign any of that logic, it was purely to move it natively into Obsidian, so ingestion doesn't require jumping out to a separate terminal step."

03 — De-risking

"Before writing any real plugin code, I ran a throwaway spike to answer the two scariest questions first: can Obsidian's renderer actually talk to Neo4j, and can it talk to Ollama? Neo4j worked over the driver's browser build. Ollama worked through Obsidian's `requestUrl` function, which routes through Electron's main process — so it sidesteps browser CORS entirely, no extra environment variables needed. Two green checks, and the spike was retired the moment its job was done."

04 — Build

"From there, four checkpoints, each one tested live inside Obsidian before moving to the next. First, the plugin scaffold and a settings tab for Neo4j and Ollama connection details. Second, vault reading — using Obsidian's own vault and metadata APIs, never touching the filesystem directly, strictly read-only. Third, extraction — calling Ollama for structured JSON, validating every triple that comes back and skipping anything malformed instead of crashing. Fourth, graph writes — idempotent MERGE operations into Neo4j, with a content hash on every note so re-running doesn't duplicate work."

05 — Caught in testing

"Running it for real surfaced three things. Ollama's JSON mode guarantees valid JSON, not the exact shape you asked for — so some empty results came back as an empty object instead of an empty array, and that's now handled gracefully instead of logged as an error. Similarly, sometimes the model returned one triple directly instead of wrapping it in an array — previously that got dropped as malformed, now it's accepted. And one bug I caught before it ever shipped: unstripped wikilinks like double-bracket Anna would've extracted as a completely different, bracketed entity that never matches the plain 'Anna' mentioned elsewhere — so I strip wikilinks before chunking, on both the note body and the title."

06 — Verified

"To prove it actually works, not just that it compiles, I ran ingestion twice back to back. The first run processed all sixteen notes and produced twenty-one entities and twenty-four mentions. The second run, with nothing edited in between, changed zero notes and wrote zero new relations — proof the content-hash skip logic is actually doing its job."

07 — Cleanup

"Once the plugin was verified, I cleaned house — removed the throwaway spike project and its installed test copy, since it had fully served its purpose. Kept the Python pipeline and its tests, the real plugin implementation, and the shared Docker and environment config that both sides rely on."

08 — What's next

"Looking ahead, the next and most important piece is contradiction resolution — right now, conflicting facts, like one note placing me in Lahore and a later one in Berlin, just sit in the graph side by side. Deciding how those resolve is genuinely the core of this whole project. After that: treating Obsidian's own wikilinks as a stronger, higher-confidence signal than LLM inference, and eventually temporal decay plus graph-based question answering with citations."

Closing

"So that's Phase 2 — the same pipeline from before, now living natively inside the vault, no terminal required. Full code's on GitHub, link on screen. Day 16, done."